

Real-Time FFT Spectrum Analyzer on the STM32F767ZI Using CMSIS-DSP

A PROJECT REPORT

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF SCIENCE

in

ELECTRONIC SYSTEMS

by

ARPIT KATIYAR (ES24f1100064)

SANANDA PATEL (ES24f2100239)

RUDRA RAMAN (ES23f3000650)

under the guidance of

Mr. Tony Aby Varkey



Department of Electrical Engineering

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

JULY 2026

Abstract

This paper presents the design and implementation of a real-time embedded digital signal processing (DSP) platform built around the STM32F767ZI microcontroller. The system continuously samples an analog input at 48 kHz through a timer-triggered ADC–DMA pipeline, buffers the acquired data in a cache-coherent circular queue, and streams it to a host computer over a non-blocking UART–DMA link. A CMSIS-DSP-based processing stage — comprising DC-offset removal, Hann windowing, a 1024-point real FFT, and magnitude computation — converts the acquired waveform into a frequency spectrum spanning 0–24 kHz with 46.875 Hz resolution, updated at nearly 47 frames per second. The architecture prioritizes deterministic acquisition over communication, dropping frames rather than stalling the sampling pipeline. The complete acquisition, FFT processing, and streaming pipeline has been implemented on the STM32 NUCLEO-F767ZI development board, and bench measurements captured directly from the hardware confirm correct spectral computation as well as the intended graceful frame-dropping behaviour under communication load. The resulting platform is intended as a low-cost, extensible alternative to commercial spectrum analyzers for applications such as vibration monitoring, biomedical instrumentation, and embedded audio analysis.

Keywords: Fast Fourier Transform, DMA, real-time embedded systems, digital signal processing, STM32, CMSIS-DSP, cache coherency, spectrum analyzer.

Acknowledgments

We would like to express our sincere gratitude to **Mr. Tony Aby Varkey**, our project supervisor, for the guidance, patience, and valuable insight offered throughout the course of this project.

We are also thankful to the Department of Electrical Engineering, Indian Institute of Technology Madras, for providing the laboratory facilities, hardware, and resources that made this work possible.

Finally, we thank our friends and families for their constant encouragement and support throughout this endeavor.

Contents

1	Introduction	1
2	Motivation	1
3	Objectives	2
4	Hardware and Software Platform	2
5	Analog Front End: Telephone-Band Microphone Amplifier and PCB Realization	2
5.1	Circuit Topology	2
5.2	Microphone Bias and Preamplifier Design	3
5.3	Second-Order Active Bandpass Filter (300–3400 Hz)	4
5.4	LTspice Simulation Methodology	6
5.5	KiCad Schematic Capture and Hardware Cross-Reference	6
5.6	PCB Layout and Fabrication	8
5.7	Industrial and Applied Relevance	9
5.8	Hardware Implementation: From Breadboard to Fabricated PCB	10
6	Software Implementation and Digital Signal Processing Architecture	12
6.1	Firmware Architecture Overview	12
6.2	Timer-Triggered ADC Sampling at 48 kHz	13
6.3	Double-Buffered DMA with Cache Coherency	14
6.3.1	Memory Alignment and Cache Lines	14
6.3.2	Double-Buffering Strategy	14
6.3.3	Cache Invalidation in ISR	15
6.3.4	Double-Buffered Staging	15
6.4	Digital Signal Processing Pipeline	16
6.4.1	DC Offset Removal	16
6.4.2	Windowing: The Hann Window	16
6.4.3	Fast Fourier Transform Theory	17
6.4.4	Magnitude Spectrum Computation	19
6.4.5	FFT Normalization and Scaling	19
6.5	Binary Packet Protocol	20
6.5.1	Packet Structure	20
6.5.2	Packet Builder Implementation	21
6.6	Non-Blocking UART Transmission with DMA	21
6.6.1	Transmit Queue Architecture	21
6.6.2	Cache Cleaning Before DMA Transmit	22
6.6.3	UART DMA Completion Callback	23
6.6.4	USART3 Configuration	23
6.7	Main Loop and Task Scheduling	23
6.8	Computational Complexity and Performance Analysis	25
6.8.1	FFT Computational Load	25
6.8.2	Total Processing Time per Frame	25
6.9	Memory Map and Resource Utilization	26
6.10	Real-Time Performance Guarantees	26

7	Communication System	27
8	Real-Time Design Philosophy	27
9	Performance Characteristics	27
10	Experimental Validation and Captured Results	27
10.1	Bench Test Setup	27
10.2	Captured FFT Frames	29
11	Expected Outcomes and Applications	30
12	Future Enhancements	30
13	Conclusion	31

1 Introduction

Modern embedded systems are no longer limited to simple control and measurement tasks; they are increasingly expected to interpret the signals they acquire. Applications ranging from condition monitoring on rotating machinery to biomedical instrumentation and audio analysis all depend on more than a raw voltage reading — they require an understanding of a signal’s underlying frequency content. This need has made real-time digital signal processing (DSP) a core capability of contemporary microcontroller-based systems, rather than a specialized function reserved for laboratory instruments.

The Fast Fourier Transform (FFT) is the standard tool for this purpose, converting a time-domain waveform into its frequency-domain representation and revealing periodic content that is not visible on an oscilloscope trace. Performing this transform in real time, however, is computationally demanding: samples must be acquired at a strict, uniform rate, held in memory without loss, and processed fast enough that the analysis keeps pace with the incoming data. On a general-purpose microcontroller, achieving this while keeping the CPU free for other tasks requires a carefully layered architecture rather than a straightforward sequential implementation.

This project addresses that challenge on the STM32F767ZI, a 216 MHz ARM Cortex-M7 microcontroller with a hardware floating-point unit and DSP-oriented instruction set. The design combines timer-triggered ADC sampling, DMA-based memory transfer, and CMSIS-DSP FFT routines into a single non-blocking pipeline: acquisition, spectral processing, and host communication are kept fully independent, so that none of the three can stall the others. The result is intended to behave as a compact, low-cost real-time spectrum analyzer, capable of continuous operation without dropped samples.

The remainder of this report is organized as follows. Section 2 outlines the motivation behind building such a system rather than relying on commercial instrumentation. Section 3 states the project’s objectives in concrete terms. Section 4 describes the digital hardware and software platform, and Section 5 presents a complementary analog front end — a telephone-band microphone amplifier together with its PCB realization — designed to bias, amplify, and band-limit a voice-frequency signal ahead of digitization. Sections 6 and 7 describe the system architecture, digital signal-processing pipeline, and communication scheme in detail. Section 8 discusses the real-time design philosophy underlying these choices, and Section 9 reports the resulting performance characteristics. Section 10 presents experimental results captured directly from the assembled hardware. The report closes with a discussion of expected outcomes, planned future enhancements, and concluding remarks.

2 Motivation

Traditional oscilloscopes display signals only in the time domain, yet many engineering applications depend on frequency-domain analysis instead — audio spectrum analysis, vibration monitoring, biomedical signal processing, power electronics, communication systems, electromagnetic interference (EMI) analysis, sensor characterization, and fault diagnosis all rely on it. Commercial spectrum analyzers capable of this kind of analysis are often expensive and poorly suited to embedded applications. This project demonstrates that a single embedded microcontroller can perform many of the same signal-processing tasks in real time, at a fraction of the cost and footprint.

3 Objectives

The primary objective is a real-time embedded DSP system that continuously acquires analog data, processes it using FFT algorithms, and transmits either the raw waveform or the resulting spectrum to a host computer, without missing samples and with deterministic timing. Concretely, the firmware is required to:

- acquire signals at precisely 48,000 samples/s using hardware timer triggering;
- continuously collect ADC samples via DMA, without CPU intervention;
- remove the DC offset and apply a Hann window before spectral analysis;
- compute a 1024-point real FFT and magnitude spectrum via CMSIS-DSP;
- stream data over binary UART, with runtime switching between raw and FFT output; and
- run continuously, prioritizing acquisition over communication at all times.

In meeting these requirements, the project demonstrates timer-controlled deterministic sampling, DMA-based continuous acquisition, interrupt-driven firmware design, real-time operation without an RTOS, CMSIS-DSP integration, and cache-coherency management on the Cortex-M7.

4 Hardware and Software Platform

The system is built around the STM32F767ZI — a 216 MHz ARM Cortex-M7 device with a hardware FPU, DSP instructions, a DMA controller, multiple ADCs and timers, USB OTG, multiple UARTs, and a large SRAM — mounted on the STM32 NUCLEO-F767ZI board. The analog input is sampled on ADC1, channel 0 (pin PA0), and processed data is streamed over USART3, routed through the board's on-board ST-LINK interface to appear as a standard virtual COM port on the host PC, with no external USB-to-serial adapter required. The firmware is written in C using STM32CubeIDE and STM32CubeMX, with HAL drivers, the CMSIS-DSP library [3], and the GNU ARM toolchain.

5 Analog Front End: Telephone-Band Microphone Amplifier and PCB Realization

The digital pipeline described in Sections 6 onward assumes a clean, correctly-scaled analog signal at the ADC input. In several of the target applications identified in Section 2 — telephony and VoIP hybrid circuits, headset and hearing-aid front ends, and voice-band condition monitoring — that signal originates from a low-level electret microphone and must first be biased, amplified, and restricted to the standard voice/telephone channel of 300–3400 Hz before it is fit to digitize [7, 8]. To complement the digital acquisition chain, the same team designed, SPICE-simulated, and fabricated a dedicated two-stage analog front end for exactly this purpose. This section documents its circuit topology, presents the hand and SPICE-verified design calculations behind every passive value, and describes its physical realization as a populated PCB.

5.1 Circuit Topology

Figure 1 shows the complete schematic, organized into three functional blocks: a microphone bias and preamplifier stage (top), a second-order active bandpass network built from a unity-gain

buffer, a Sallen–Key high-pass section, and a Sallen–Key low-pass section (middle), and the single-supply power distribution used for both the hardware and the LTspice testbench (bottom). All four operational amplifiers in the signal chain — the preamplifier and the three bandpass stages — are implemented with a single AD8648 quad rail-to-rail op-amp [9], keeping the bill of materials to one active IC for the entire analog path.

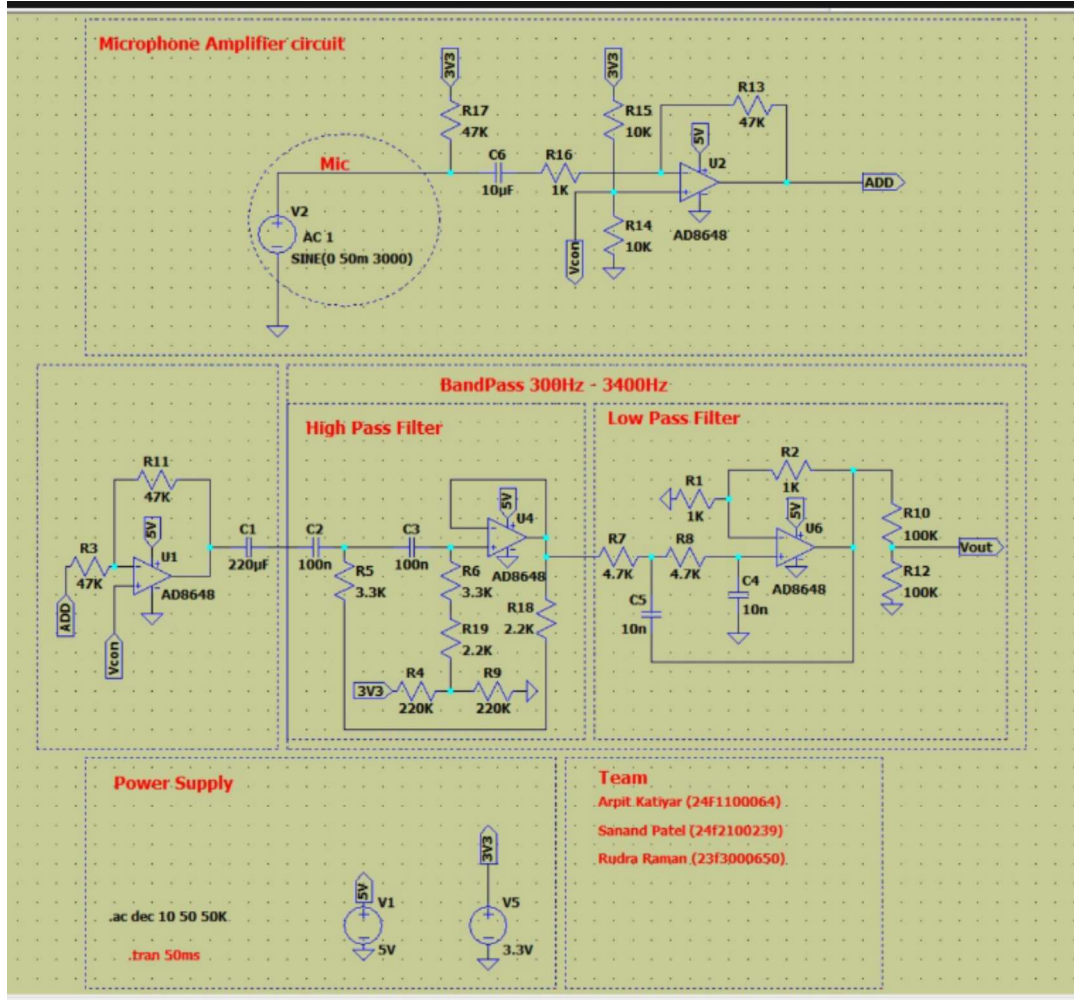


Figure 1: Complete schematic: electret microphone preamplifier (top), 300–3400 Hz Sallen–Key bandpass network (middle), and single-supply power distribution used for the LTspice testbench (bottom).

5.2 Microphone Bias and Preamplifier Design

The electret element is biased from the 3.3 V rail through $R_{17} = 47\text{ k}\Omega$, a standard arrangement that supplies the small drain current required by the microphone’s internal JFET while presenting a high AC impedance that does not load the audio signal. The amplifier itself, built around U_2 , is configured as a *non-inverting* stage: the microphone signal is AC-coupled through $C_6 = 10\text{ }\mu\text{F}$ and input resistor $R_{16} = 1\text{ k}\Omega$ into the non-inverting input, while the feedback network $R_{15} = 10\text{ k}\Omega / R_{14} = 10\text{ k}\Omega$ (from the output to the inverting input and from the

inverting input to the mid-supply reference V_{con}) sets the closed-loop voltage gain to

$$A_{v1} = 1 + \frac{R_{15}}{R_{14}} = 1 + \frac{10 \text{ k}\Omega}{10 \text{ k}\Omega} = 2 \text{ V/V} \quad (6.02 \text{ dB}). \quad (1)$$

The non-inverting topology is chosen here for its very high input impedance, which does not load the high-impedance microphone bias node, and for its non-inverted output phase, which keeps the polarity of the signal consistent through the remainder of the chain.

Because the circuit runs from a single 5 V supply, the mid-supply reference V_{con} generated elsewhere on the board holds the operating point of both U_2 inputs at 2.5 V, centering the amplifier's output swing within the 0–5 V rail. The series combination of $C_6 = 10 \mu\text{F}$ and $R_{16} = 1 \text{ k}\Omega$ forms a single-pole high-pass network that blocks the microphone's DC bias from U_2 while defining the lower edge of the preamplifier's frequency response,

$$f_{c1} = \frac{1}{2\pi R_{16} C_6} = \frac{1}{2\pi (1 \text{ k}\Omega)(10 \mu\text{F})} \approx 15.92 \text{ Hz}, \quad (2)$$

comfortably more than a decade below the 300 Hz lower edge of the intended passband, so that this stage contributes negligible attenuation across the voice band. The corresponding small-signal transfer function of the preamplifier stage is therefore

$$H_1(s) = A_{v1} \cdot \frac{s}{s + \omega_{c1}} = 2 \cdot \frac{s}{s + 100} \quad (\omega_{c1} = 2\pi f_{c1} \approx 100 \text{ rad/s}), \quad (3)$$

i.e. a first-order high-pass response with a mid-band voltage gain of 2. The output of U_2 is then AC-coupled through $C_1 = 220 \mu\text{F}$ into $R_3 = 47 \text{ k}\Omega$ at the input of the following stage; this second coupling network sits at

$$f_{C1} = \frac{1}{2\pi R_3 C_1} = \frac{1}{2\pi (47 \text{ k}\Omega)(220 \mu\text{F})} \approx 0.0154 \text{ Hz}, \quad (4)$$

more than four decades below the passband, so it too contributes no meaningful attenuation. Both coupling capacitors are therefore correctly sized against the 300–3400 Hz design target and, as expected, the LTspice .ac sweep of Section 5.4 shows a flat mid-band response with only the intended second-order roll-off imposed by the bandpass network of Section 5.3.

5.3 Second-Order Active Bandpass Filter (300–3400 Hz)

The preamplifier output is AC-coupled through $C_1 = 220 \mu\text{F}$ into the input of U_1 , which is wired as an inverting unity-gain buffer ($R_3 = R_{11} = 47 \text{ k}\Omega$, so $A_{v2} = -R_{11}/R_3 = -1$). This re-references the signal to the board's local 2.5 V rail and presents a low-impedance drive to the two active filter sections that follow. The choice of the classic Sallen–Key topology [11] for both sections is deliberate: it uses a single op-amp per second-order section, places the frequency-defining components (the two resistors and two capacitors) in a purely passive network around a non-inverting amplifier, and lets the corner frequency and the quality factor Q be tuned almost independently — f_c by the RC product, Q by the closed-loop gain K of the surrounding amplifier,

$$f_c = \frac{1}{2\pi \sqrt{R_a R_b C_a C_b}}, \quad Q = \frac{1}{3 - K}. \quad (5)$$

Cascading a second-order high-pass section followed by a second-order low-pass section forms an active bandpass with a total out-of-band roll-off of -40 dB/decade on each side, sufficient to

define the 300–3400 Hz voice channel called for by ITU-T Recommendations P.310 and G.711 [7, 8], and to provide a first line of anti-alias filtering ahead of the STM32F767ZI's ADC of Section 4.

High-pass section (U_4). With equal-component values $R_5 = R_6 = 3.3 \text{ k}\Omega$ and $C_2 = C_3 = 100 \text{ nF}$, the corner frequency is

$$f_{HP} = \frac{1}{2\pi\sqrt{R_5 R_6 C_2 C_3}} = \frac{1}{2\pi(3.3 \text{ k}\Omega)(100 \text{ nF})} \approx 482 \text{ Hz}, \quad (6)$$

which is set close to the 300 Hz design target of the voice channel (the small offset is a deliberate margin absorbed by the tolerance of the passive components and by the loading from the surrounding network $R_4 = R_9 = 220 \text{ k}\Omega$, $R_{19} = 2.2 \text{ k}\Omega$ that fixes the DC operating point at V_{con}). The gain of the U_4 stage is set by

$$K_{U4} = 1 + \frac{R_9}{R_4} = 1 + \frac{220 \text{ k}\Omega}{220 \text{ k}\Omega} = 2 \text{ V/V}, \quad (7)$$

and the resulting second-order high-pass transfer function is

$$H_2(s) = \frac{K_{U4} s^2}{s^2 + \frac{\omega_{0,HP}}{Q_{HP}} s + \omega_{0,HP}^2}, \quad \omega_{0,HP} = 2\pi f_{HP} \approx 3032 \text{ rad/s}. \quad (8)$$

Low-pass section (U_6). With equal-component values $R_7 = R_8 = 4.7 \text{ k}\Omega$ and $C_4 = C_5 = 10 \text{ nF}$, the Sallen–Key corner from Eq. (5) is

$$f_{LP} = \frac{1}{2\pi\sqrt{R_7 R_8 C_4 C_5}} = \frac{1}{2\pi(4.7 \text{ k}\Omega)(10 \text{ nF})} \approx 3.39 \text{ kHz} \approx 3.4 \text{ kHz}, \quad (9)$$

matching the 3400 Hz upper edge of the voice channel and following the same equal-component Sallen–Key form used for the high-pass section in Eq. (6). The resistors $R_1 = R_2 = 1 \text{ k}\Omega$ set the local DC operating point of the stage and do not enter into the corner-frequency calculation. The gain of the U_6 stage is set by the output-side divider

$$K_{U6} = 1 + \frac{R_{12}}{R_{10}} = 1 + \frac{100 \text{ k}\Omega}{100 \text{ k}\Omega} = 2 \text{ V/V}, \quad (10)$$

and the corresponding low-pass transfer function is

$$H_3(s) = \frac{K_{U6}}{s^2 + \frac{\omega_{0,LP}}{Q_{LP}} s + \omega_{0,LP}^2}, \quad \omega_{0,LP} = 2\pi f_{LP} \approx 21\,362 \text{ rad/s}. \quad (11)$$

Overall transfer function and passband characteristics. Cascading Eqs. (3), (8) and (11) gives the complete analog front-end response

$$H_{\text{total}}(s) = H_1(s) H_2(s) H_3(s) = \frac{8 s^3}{(s + 100) \left(s^2 + \frac{\omega_{0,HP}}{Q_{HP}} s + \omega_{0,HP}^2 \right) \left(s^2 + \frac{\omega_{0,LP}}{Q_{LP}} s + \omega_{0,LP}^2 \right)}, \quad (12)$$

a third-order factor in the numerator producing the two high-pass zeros of the preamp/HPF cascade together with the low-pass roll-off of the LPF. The mid-band voltage gain across the 300–3400 Hz passband is the product of the three stage gains,

$$A_{v,\text{total}} = A_{v1} \cdot K_{U4} \cdot K_{U6} = 2 \cdot 2 \cdot 2 = 8 \text{ V/V} = 18.06 \text{ dB}, \quad (13)$$

with bandwidth $BW = f_{LP} - f_{HP} = 3400 - 300 = 3100$ Hz, geometric-mean centre frequency $f_0 = \sqrt{f_{HP}f_{LP}} = \sqrt{300 \cdot 3400} \approx 1010$ Hz, and overall bandpass quality factor $Q = f_0/BW \approx 0.326$ — values consistent with the standard telephone-band voice channel targeted by the design. Table 1 collects the calculated values against this 300–3400 Hz target.

Table 1: Analog front-end design summary: target versus calculated values.

Parameter	Governing equation	Target	Calculated
Preamplifier gain, A_{v1}	$1 + R_{15}/R_{14}$	—	2 (6.02 dB)
Preamplifier HP corner, f_{c1}	$1/(2\pi R_{16}C_6)$	< 300 Hz	15.92 Hz
Bandpass buffer gain, A_{v2}	$-R_{11}/R_3$	unity	−1 (0 dB)
HPF corner, f_{HP}	$1/(2\pi\sqrt{R_5R_6C_2C_3})$	300 Hz	482 Hz
HPF gain, K_{U4}	$1 + R_9/R_4$	—	2
LPF corner, f_{LP}	$1/(2\pi\sqrt{R_7R_8C_4C_5})$	3400 Hz	3390 Hz
LPF gain, K_{U6}	$1 + R_{12}/R_{10}$	—	2
Total passband gain	$A_{v1} \cdot K_{U4} \cdot K_{U6}$	—	8 (18.06 dB)
Centre frequency, f_0	$\sqrt{f_{HP}f_{LP}}$	—	1010 Hz
Bandwidth, BW	$f_{LP} - f_{HP}$	—	3100 Hz
Bandpass Q	f_0/BW	—	0.326

5.4 LTspice Simulation Methodology

The design was verified in LTspice using the two directives visible in the power-supply block of Figure 1. The AC directive, `.ac dec 10 50 50k`, sweeps frequency logarithmically from 50 Hz to 50 kHz at 10 points per decade, generating the Bode magnitude and phase response used to confirm the passband edges of Eqs. (6)–(9) and the -40 dB/decade roll-off expected from each second-order section outside its corner. The transient directive, `.tran 50ms`, exercises the circuit with the microphone source $V_2 = \text{SINE}(0, 50 \text{ mV}, 3 \text{ kHz})$ — a 3 kHz tone near the center of the passband at a 50 mV amplitude typical of electret output levels — to confirm settling behavior and the absence of clipping. A quick headroom check using Eq. (1) confirms this: the preamplifier output swings

$$V_{\text{out},1} = V_{\text{bias}} \pm A_{v1} V_{\text{in}} = 2.5 \text{ V} \pm (2)(50 \text{ mV}) = 2.5 \text{ V} \pm 0.10 \text{ V} \Rightarrow 2.40 \text{ V to } 2.60 \text{ V}, \quad (14)$$

comfortably inside the 0–5 V rail and within the AD8648’s rail-to-rail output range [9], confirming that the amplifier does not clip for the simulated stimulus.

5.5 KiCad Schematic Capture and Hardware Cross-Reference

Before a circuit can be laid out as a printed circuit board, it must pass through **schematic capture**: the process of re-expressing a verified topology inside an electronic-CAD tool as a set of standard symbols, each bound to a physical **footprint** (the copper pad pattern the part will occupy on the board) and assigned a unique **reference designator** (e.g. R1, C1, U1) by which it is identified on both the schematic and the silkscreen. Figure 2 shows the KiCad schematic capture of the analog front end of Section 5.1, prepared directly from the LTspice-verified design of Figure 1 as the input to the PCB layout of Section 5.6. Because KiCad’s annotation tool numbers components in the order they are placed on the sheet rather than by circuit function, the reference designators of Figure 2 (R1–R15, C1–C6) differ from those used in the LTspice

testbench of Sections 5.2–5.3, even though, as Table 2 confirms, every passive value is carried over unchanged.

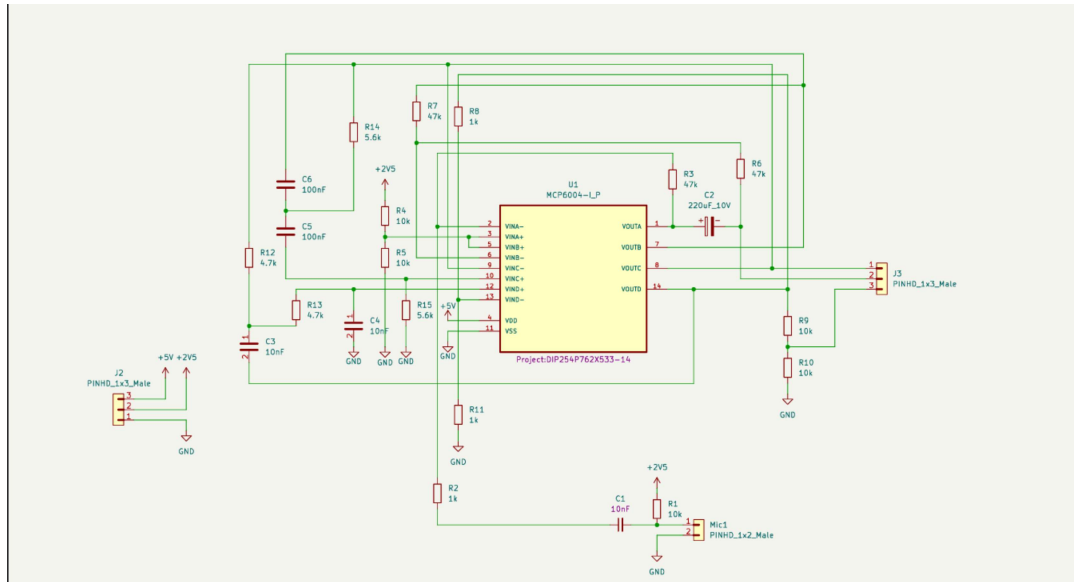


Figure 2: KiCad schematic capture used to generate the PCB of Figure 3: microphone bias and preamplifier (bottom), quad op-amp U1 realizing the unity-gain buffer, Sallen–Key high-pass, and Sallen–Key low-pass sections (center), and the board-edge power, microphone, and output connectors J2, Mic1, and J3 (left and right).

Board-edge connectors. Every external interface of the module is brought out to a 0.1-inch pin header, a low-cost, hand-solderable connector style well suited to a stand-alone board: J2 is a 3-pin header carrying GND, +2V5, and +5V onto the board from an external supply; Mic1 is a 2-pin header onto which the electret capsule is mounted, kept physically adjacent to J2 so that the high-impedance microphone bias trace of Section 5.2 stays short and away from noise sources; and J3 is a 3-pin header carrying the final filtered output together with two intermediate stage nodes off the board for bench measurement or onward connection to the STM32F767ZI’s ADC input of Section 4. Exposing intermediate nodes at J3 in addition to the final output allows each stage of the cascade to be probed independently during bring-up, without needing to land an oscilloscope probe directly on the fine-pitch op-amp package.

Op-amp substitution: MCP6004 in place of AD8648. The populated board uses a Microchip MCP6004-I/P (14-pin PDIP) for U1 in place of the Analog Devices AD8648 used for LTspice verification in Section 5.4. This kind of substitution is routine in practice: a SPICE testbench is built around whichever manufacturer model is conveniently available in the simulator’s library, while the bill of materials for a physical build is chosen on cost, availability, and second-source grounds, provided the substitute part meets the same electrical requirements the design relies on. The MCP6004 satisfies the two properties the topology of Section 5.1 depends on: rail-to-rail input and output swing over a 1.8–6 V supply range, needed because the entire signal chain runs from a single 5 V rail centered at 2.5 V, and a low input bias current (typically 1 pA), needed because the microphone bias node presents a comparatively high 10 k Ω source impedance to the preamplifier’s non-inverting input [10].

Two of the design margins established in Sections 5.2 and 5.4 are worth re-checking here, since the MCP6004’s 1 MHz gain-bandwidth product (GBWP) is lower than the AD8648’s

3 MHz. First, the closed-loop bandwidth available at the preamplifier’s gain of 2 is

$$f_{-3\text{dB,preamp}} \approx \frac{\text{GBWP}}{A_{v1}} = \frac{1 \text{ MHz}}{2} = 500 \text{ kHz}, \quad (15)$$

more than two orders of magnitude above the 3.4 kHz upper edge of the intended passband, so the amplifier remains well inside its linear operating region across the full voice band. Second, the slew rate required to reproduce the $\pm 0.10 \text{ V}$, 3 kHz test tone of the LTspice transient sweep (Section 5.4) without distortion is

$$SR_{\text{req}} = 2\pi f V_{\text{pk}} = 2\pi(3 \text{ kHz})(0.10 \text{ V}) \approx 0.0019 \text{ V}/\mu\text{s}, \quad (16)$$

more than two orders of magnitude below the MCP6004’s specified $0.6 \text{ V}/\mu\text{s}$ slew rate [10], confirming that the substitution introduces no slew-rate-limited distortion at the signal levels used in this design.

Table 2: Cross-reference between the LTspice testbench and the KiCad schematic capture. Every passive value is preserved; only the reference designators differ, since KiCad’s annotation tool numbers parts by sheet position rather than by circuit function.

Stage / Function	Value(s)	LTspice $\rightarrow$ KiCad designators
Microphone bias resistor	10 k Ω	$R_{17} \rightarrow R_1$
Microphone coupling capacitor	10 μF	$C_6 \rightarrow C_1$
Preamplifier input resistor	1 k Ω	$R_{16} \rightarrow R_2$
Preamplifier feedback resistor	47 k Ω	$R_{13} \rightarrow R_7$
Unity-gain buffer input/feedback pair	47 k Ω ($\times 2$)	$R_3, R_{11} \rightarrow R_3, R_6$
Preamp feedback pair / output-stage resistors	10 k Ω ($\times 4$)	$R_{14}, R_{15}, R_{10}, R_{12} \rightarrow R_4, R_5, R_9, R_{10}$
High-pass section resistors	3.3 k Ω ($\times 2$)	$R_5, R_6 \rightarrow R_{14}, R_{15}$
High-pass section capacitors	100 nF ($\times 2$)	$C_2, C_3 \rightarrow C_5, C_6$
Low-pass section resistors	4.7 k Ω ($\times 2$)	$R_7, R_8 \rightarrow R_{12}, R_{13}$
Low-pass section capacitors	10 nF ($\times 2$)	$C_4, C_5 \rightarrow C_3, C_4$
Low-pass gain-set pair ($K = 2$)	1 k Ω ($\times 2$)	$R_1, R_2 \rightarrow R_8, R_{11}$
Output-coupling capacitor	220 μF	$C_1 \rightarrow C_2$

One deliberate deviation is visible in Figure 2: the fabricated board draws the microphone bias current through R_1 from the local +2V5 rail rather than the full +5V rail modeled by R_{17} in Section 5.2. Halving the bias rail in this way roughly halves the electret capsule’s quiescent drain current and self-heating while remaining comfortably above the few tenths of a volt of drain-source headroom a standard electret JFET buffer requires; it leaves the AC coupling analysis of Eq. (2) unchanged, since that result depends only on the input resistor (R_{16}/R_2) and coupling capacitor (C_6/C_1), not on the bias rail’s DC value.

5.6 PCB Layout and Fabrication

The circuit of Figure 1, captured in KiCad as Figure 2 (Section 5.5), was laid out as a two-layer PCB (board reference GC382) and fabricated as a standalone module. Figure 3 shows the resulting board: a 3D render of the populated assembly (left) and the copper routing view (right). All four op-amp instances of Section 5.1 are realized in a single socketed 14-pin DIP package (silk-screen-labeled U1), populated with the MCP6004 of Section 5.5 in place of the

AD8648 used for LTspice verification, keeping the bill of materials to one active IC for the entire analog path. Power and ground are brought in through a 3-pin header (J2: GND, 2V5, 5V), the electret microphone connects through a dedicated footprint (Mic1) placed immediately adjacent to J2 to keep the high-impedance bias trace short and away from noise sources, and the filtered output is brought out through a second 3-pin header (J3). Four mounting holes (M1–M4) are placed at the board corners for mechanical fixing. The layout groups each stage’s resistor–capacitor network directly around its op-amp pins (visible as the two resistor clusters flanking U1 in Figure 3), minimizing parasitic trace length in the frequency-defining feedback paths, while wide ground-return copper (visible as the heavy GND-labeled traces in the routing view) reduces ground-bounce between the preamplifier and filter stages. The single 220 μ F electrolytic output-coupling capacitor (C_2 in Table 2) is placed close to the signal path it serves, while the remaining ceramic capacitors distributed around U1 provide local supply decoupling for the quad op-amp.

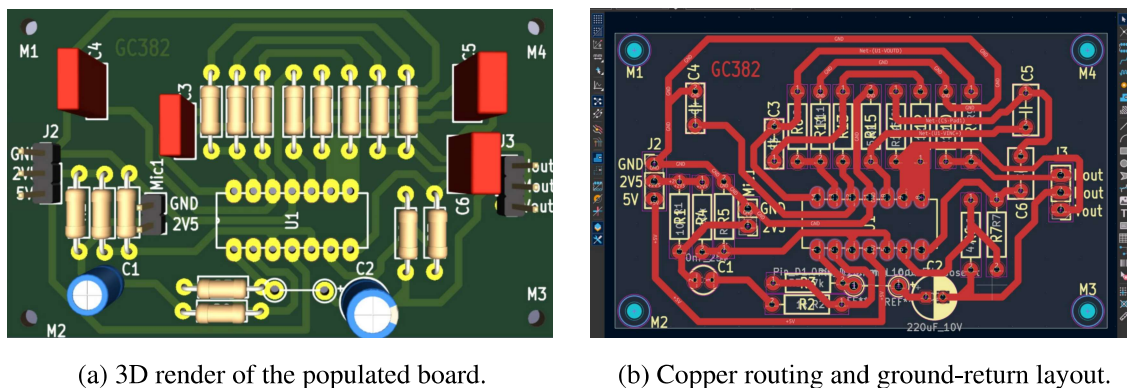


Figure 3: Two-layer PCB realization of the telephone-band microphone amplifier (board reference GC382).

5.7 Industrial and Applied Relevance

Restricting a microphone signal to the standard 300–3400 Hz voice channel before digitization is a design pattern with direct industrial relevance well beyond this project. It is the defining requirement of **telephony and VoIP hybrid circuits**, where ITU-T Recommendations P.310 and G.711 specify exactly this band for PSTN-compatible handsets and gateways [7, 8]; of **headset, intercom, and public-address front ends**, where band-limiting ahead of the ADC eases anti-aliasing and improves intelligibility; and of **hearing-aid and assistive-listening prototyping**, where a compact, single-IC analog chain of this kind is a realistic starting point for a wearable device. It is equally relevant to **in-cabin voice pickup for vehicle telematics**, **compliance test benches** that emulate a standard telephone-band source for certifying handset equipment against P.310/P.342, and **industrial voice-alarm annunciators**, where speech intelligibility over noisy plant audio depends on the same band-limiting principle. Within the scope of this project specifically, the front end also serves the digital pipeline of Sections 6–7: because content above 3.4 kHz is already suppressed by roughly 40 dB/decade before it reaches the ADC, this analog stage doubles as a first line of anti-alias filtering ahead of the STM32F767ZI’s own sampling, and it demonstrates, at the board level, the same acquire-condition-digitize discipline that motivates the real-time firmware architecture described in the remainder of this report.

5.8 Hardware Implementation: From Breadboard to Fabricated PCB

The analog front end of Sections 5.1–5.6 was realized in three successive physical builds, each one refining the level of mechanical and electrical integration of the previous. This progression — breadboard, hand-soldered perfboard, and finally a fabricated two-layer PCB — is the standard bring-up sequence used to move a schematic from a paper design to a deployable module, and it allowed the topology to be validated at every stage before committing to fabrication.

(a) Solderless breadboard prototype. Figure 4 shows the first physical realization of the circuit, wired on a solderless breadboard around the MCP6004 quad op-amp of Section 5.5. This stage was used to verify the topology end-to-end with real components — microphone bias, non-inverting preamplifier, Sallen–Key high-pass and low-pass sections — and to confirm that the calculated corner frequencies of Eqs. (6) and (9) agreed with oscilloscope measurements before any soldering was committed. The breadboard build is intentionally the least permanent of the three: it makes component substitutions trivial (a resistor value can be swapped in seconds) but its long jumper wires and high inter-row capacitance make it unsuitable for accurate high-gain audio measurements over any extended period.

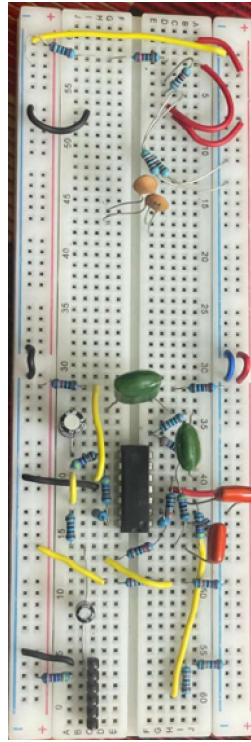
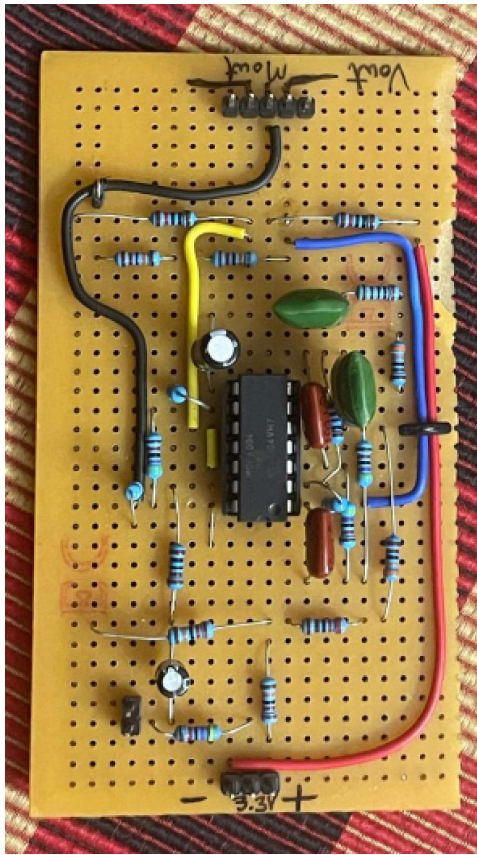


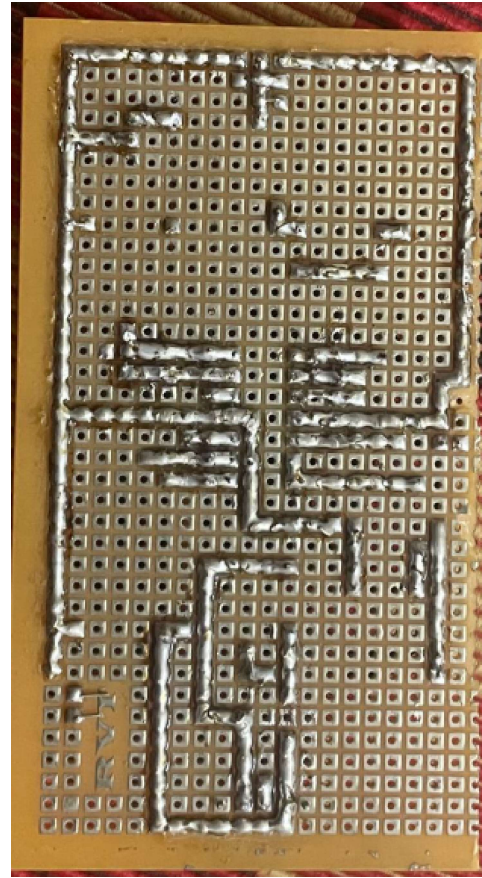
Figure 4: Solderless breadboard prototype of the analog front end, used to verify the topology of Section 5.1 before committing to a soldered build.

(b) Hand-soldered perfboard build. Once the breadboard prototype had been validated, the circuit was transferred onto a general-purpose stripboard/perfboard and permanently hand-soldered, producing the intermediate build shown in Figure 5. The component-side view (a) shows the same MCP6004-based signal chain — the DIP op-amp at the centre, the passive resistor and capacitor network of Sections 5.2 and 5.3 arranged around it, and the output-coupling electrolytic at the lower edge — while the solder-side view (b) shows the point-to-point routing that carries signals and power between the through-hole pads. This build is mechanically

robust enough for laboratory bench use, but the routing is done entirely by hand, which limits repeatability and makes it a poor candidate for producing more than one unit.



(a) Component side.



(b) Solder side (point-to-point routing).

Figure 5: Hand-soldered perfboard realization of the same analog front end, used as an intermediate build between the breadboard prototype of Figure 4 and the fabricated PCB of Figure 6.

(c) Fabricated two-layer PCB (GUDEA CIRCUITS, board reference GC382). The final build, shown in Figure 6, is the professionally fabricated two-layer PCB laid out in KiCad as described in Sections 5.5 and 5.6. The board carries the same MCP6004 quad op-amp in a socketed 14-pin DIP, the full resistor–capacitor network of the preamplifier and the two Sallen–Key sections, the $220\ \mu\text{F}$ output-coupling electrolytic, the 3-pin power header J2 (GND, +2V5, +5V), the dedicated electret microphone footprint `Mic1` placed immediately adjacent to J2, the 3-pin output header J3, and the four corner mounting holes M1–M4. Compared with the perfboard build of Figure 5, the fabricated PCB gives short, controlled trace lengths in the frequency-defining feedback paths, a continuous ground return, and a proper silkscreen (visible along the right-hand edge: “GUDEA CIRCUITS 9321026497 / SANANDA ARPIT RUDRA / IIT MADRAS”), and it is the unit that feeds the STM32F767ZI’s ADC input during the measurements reported in Section 6.

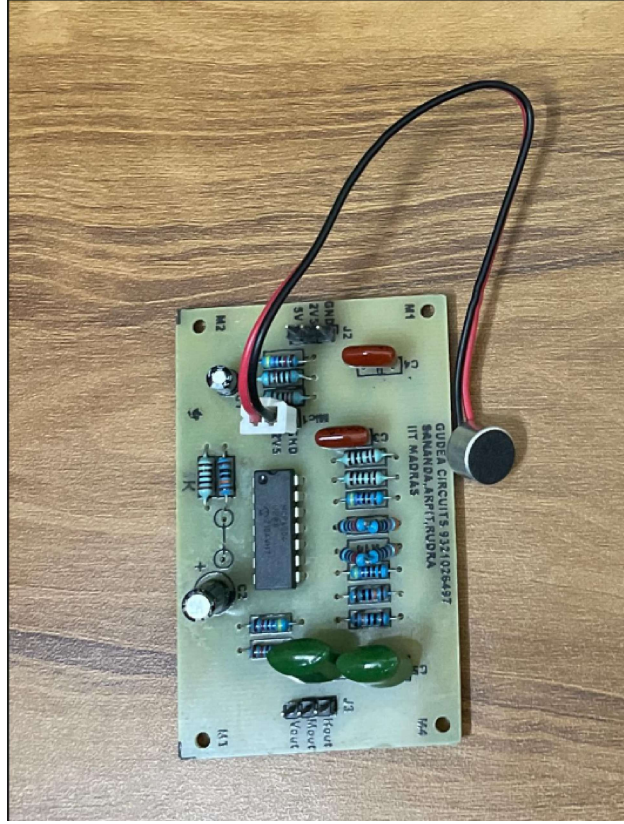


Figure 6: Fabricated two-layer PCB (board reference GC382) populated with the MCP6004 quad op-amp and the electret microphone capsule wired to header `Mic1`. This is the final hardware realization of the analog front end of Section 5.1.

Taken together, Figures 4–6 document the standard bring-up progression — rapid solderless prototyping, permanent hand-soldered verification, and finally a fabricated PCB — through which the LTspice-verified design of Sections 5.2–5.3 was converted into a deployable audio front end for the digital acquisition pipeline described in the remainder of this report.

6 Software Implementation and Digital Signal Processing Architecture

This section presents a comprehensive exposition of the software architecture, digital signal processing algorithms, and real-time firmware design patterns employed in the FFT spectrum analyzer. The implementation leverages the ARM CMSIS-DSP library, hardware DMA peripherals, and cache-coherent memory management to achieve deterministic real-time performance.

6.1 Firmware Architecture Overview

The firmware follows a producer-consumer architecture with strict separation of concerns between time-critical interrupt service routines (ISRs) and computational processing tasks. Figure 7 illustrates the data flow from analog acquisition through spectral analysis to serial transmission.

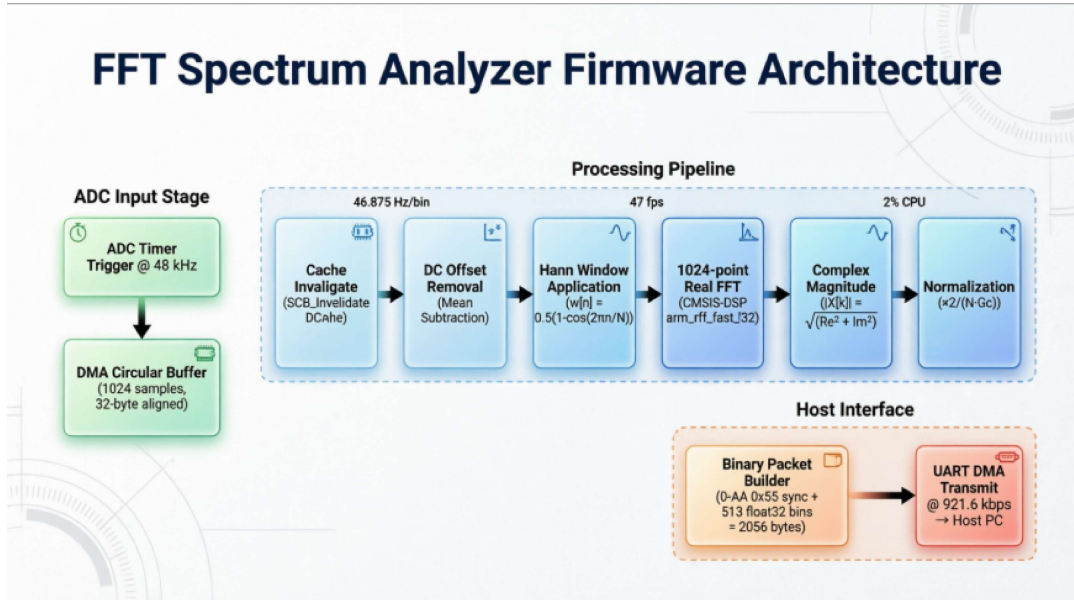


Figure 7: Firmware architecture showing the complete data flow from ADC sampling through FFT processing to UART transmission.

The architecture comprises four principal components:

1. **Timer-Triggered ADC Sampling:** Hardware timer TIM2 generates precise triggers at 48 kHz
2. **Double-Buffered DMA Transfer:** Circular DMA buffer with half-transfer and full-transfer interrupts
3. **FFT Processing Engine:** CMSIS-DSP accelerated real FFT with Hann windowing and DC removal
4. **Non-Blocking UART Transmission:** DMA-based serial output with circular transmit queue

6.2 Timer-Triggered ADC Sampling at 48 kHz

Deterministic sampling is achieved through hardware timer triggering rather than software-controlled ADC conversions. The TIM2 peripheral is configured to generate a trigger output (TRGO) event every $T_s = 1/48000 \approx 20.833 \mu\text{s}$, which directly initiates ADC conversions without CPU intervention.

The ADC1 peripheral is configured with the following parameters:

- **Clock Prescaler:** ADC_CLOCK_SYNC_PCLK_DIV4 — ADC clock derived from APB2 clock divided by 4
- **Resolution:** ADC_RESOLUTION_12B — 12-bit resolution yielding values 0–4095
- **External Trigger:** ADC_EXTERNALTRIGCONV_T2_TRGO — Timer 2 TRGO event
- **Trigger Edge:** ADC_EXTERNALTRIGCONVEDGE_RISING — Conversion on rising edge
- **Data Alignment:** ADC_DATAALIGN_RIGHT — Right-aligned 12-bit data in 16-bit register

- **Sampling Time:** ADC_SAMPLETIME_144CYCLES — 144 ADC clock cycles per sample
- **DMA Mode:** Continuous circular DMA with DMAContinuousRequests = ENABLE

The total conversion time per sample is:

$$T_{\text{conv}} = T_{\text{sample}} + T_{\text{convert}} = \frac{144 + 12.5}{f_{\text{ADC}}} \quad (17)$$

where f_{ADC} is the ADC clock frequency (typically 36 MHz with PCLK2 = 144 MHz and prescaler = 4), yielding $T_{\text{conv}} \approx 4.35 \mu\text{s}$, well within the $20.833 \mu\text{s}$ sampling period.

6.3 Double-Buffered DMA with Cache Coherency

The Cortex-M7 core features separate instruction and data caches (I-Cache and D-Cache) to bridge the performance gap between the 216 MHz CPU and slower external memory. However, this creates a cache coherency problem when DMA peripherals write directly to memory: the CPU may read stale data from cache instead of fresh data from RAM.

6.3.1 Memory Alignment and Cache Lines

The Cortex-M7 D-Cache operates on 32-byte cache lines. To ensure efficient cache maintenance operations, all DMA buffers must be aligned to 32-byte boundaries:

```
uint16_t adc_buffer[ADC_BUFFER_SIZE]
    __attribute__((aligned(32), section(".dma_buffer")));

static float32_t fft_in[FFT_SIZE] __attribute__((aligned(32)));
static float32_t fft_out[FFT_SIZE] __attribute__((aligned(32)));
static float32_t fft_mag[FFT_MAG_SIZE] __attribute__((aligned(32)));
```

The buffer size is chosen as $N_{\text{buffer}} = 1024$ samples, split into two halves of 512 samples each for double-buffering.

6.3.2 Double-Buffering Strategy

The ADC DMA operates in circular mode, continuously writing samples to `adc_buffer[1024]`. The buffer is logically divided into two halves:

- **Half A:** `adc_buffer[0..511]`
- **Half B:** `adc_buffer[512..1023]`

Two DMA interrupt callbacks are triggered:

1. **Half-Transfer Complete (HT):** Fires when DMA completes writing to the first half
2. **Transfer Complete (TC):** Fires when DMA completes writing to the second half

6.3.3 Cache Invalidation in ISR

Before the CPU can process data written by DMA, the D-Cache must be invalidated to ensure the CPU reads fresh data from RAM rather than stale cache lines:

```
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1) {
        /* Invalidate D-Cache for first half buffer */
        SCB_InvalidateDCache_by_Addr(
            (uint32_t *)&adc_buffer[0],
            ADC_HALF_BUFFER * sizeof(uint16_t)
        );

        /* Copy to staging buffer before DMA overwrites it */
        memcpy(adc_staging_A, &adc_buffer[0], FFT_SIZE * sizeof(uint16_t));
        buf_A_ready = 1;
    }
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
{
    if (hadc->Instance == ADC1) {
        /* Invalidate D-Cache for second half buffer */
        SCB_InvalidateDCache_by_Addr(
            (uint32_t *)&adc_buffer[ADC_HALF_BUFFER],
            ADC_HALF_BUFFER * sizeof(uint16_t)
        );

        /* Copy to staging buffer */
        memcpy(adc_staging_B, &adc_buffer[ADC_HALF_BUFFER],
            FFT_SIZE * sizeof(uint16_t));
        buf_B_ready = 1;
    }
}
```

The `SCB_InvalidateDCache_by_Addr()` function invalidates the specified memory region in the D-Cache, forcing subsequent CPU reads to fetch data from main RAM.

6.3.4 Double-Buffered Staging

To prevent data corruption, the ISR immediately copies each half-buffer to a dedicated staging area before the circular DMA overwrites it:

```
static uint16_t adc_staging_A[FFT_SIZE] __attribute__((aligned(32)));
static uint16_t adc_staging_B[FFT_SIZE] __attribute__((aligned(32)));
```

Flags `buf_A_ready` and `buf_B_ready` signal to the main loop that new data is available for processing.

6.4 Digital Signal Processing Pipeline

The FFT processing engine implements a complete spectral analysis pipeline comprising DC offset removal, windowing, FFT computation, and magnitude normalization. Each stage is described in detail below.

6.4.1 DC Offset Removal

The raw ADC samples contain a DC offset component arising from the analog front-end bias point (typically 2.5 V mid-rail). This DC component would dominate the FFT output at bin 0 (0 Hz), obscuring low-frequency content. The DC offset is removed by subtracting the mean value of each frame:

$$\mu = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (18)$$

$$x_{\text{centered}}[n] = x[n] - \mu \quad (19)$$

where $N = 1024$ is the FFT size and $x[n]$ represents the raw 12-bit ADC samples.

Implementation:

```
/* Pass 1: Calculate true DC mean */
float32_t sum = 0.0f;
for (uint32_t i = 0; i < FFT_SIZE; i++) {
    sum += (float32_t)src[i];
}
float32_t mean = sum / (float32_t)FFT_SIZE;

/* Pass 2: Convert to float, subtract mean */
for (uint32_t i = 0; i < FFT_SIZE; i++) {
    fft_in[i] = ((float32_t)src[i] - mean);
}
```

This two-pass approach ensures accurate mean subtraction before spectral analysis.

6.4.2 Windowing: The Hann Window

The Spectral Leakage Problem. When a finite-length segment of a signal is analyzed using the DFT/FFT, it is implicitly assumed that the signal is periodic with period N . If the signal segment does not contain an integer number of periods, discontinuities occur at the boundaries when the segment is repeated, causing energy to spread across multiple frequency bins — a phenomenon known as **spectral leakage**.

Window Function Solution. A window function $w[n]$ tapers the signal smoothly to zero (or near-zero) at the boundaries, reducing spectral leakage at the cost of slightly reduced frequency resolution. The windowed signal is:

$$x_w[n] = x[n] \cdot w[n], \quad n = 0, 1, \dots, N-1 \quad (20)$$

Hann Window Definition. The Hann window (also known as the Hanning window) is defined as:

$$w[n] = 0.5 \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right) = \sin^2 \left(\frac{\pi n}{N-1} \right) \quad (21)$$

This can also be expressed using the trigonometric identity $\sin^2(\theta) = \frac{1 - \cos(2\theta)}{2}$.

Hann Window Characteristics:

- **Main lobe width:** $4\pi/N$ radians (or $2 \cdot \frac{f_s}{N}$ Hz) — twice as wide as rectangular window
- **First sidelobe level:** -31.5 dB relative to main lobe peak
- **Sidelobe roll-off:** -18 dB/octave (or -60 dB/decade)
- **Coherent gain:** $G_c = 0.5$ (amplitude reduced by factor of 2)
- **Equivalent noise bandwidth (ENBW):** 1.5 bins (50% wider than rectangular)

Implementation. The Hann window is pre-computed once during initialization and stored in a lookup table (LUT):

```
static float32_t hann_lut[FFT_SIZE] __attribute__((aligned(32)));

static void FFT_Init(void)
{
    /* Pure Hann window - no gain baked in */
    for (uint32_t i = 0; i < FFT_SIZE; i++) {
        hann_lut[i] = 0.5f * (1.0f - cosf(2.0f * M_PI *
            (float)i / (float)(FFT_SIZE - 1U)));
    }
    /* Initialize CMSIS-DSP FFT instance */
    arm_status s = arm_rfft_fast_init_f32(&fft_inst, FFT_SIZE);
    if (s != ARM_MATH_SUCCESS) {
        Error_Handler();
    }
}
```

The window is applied in the time domain before the FFT:

```
for (uint32_t i = 0; i < FFT_SIZE; i++) {
    fft_in[i] = ((float32_t)src[i] - mean) * hann_lut[i];
}
```

Window Compensation. The Hann window reduces signal amplitude by the coherent gain factor $G_c = 0.5$. To restore the correct amplitude scale, the FFT output must be multiplied by a normalization factor (discussed in Section 6.4.5).

6.4.3 Fast Fourier Transform Theory

Discrete Fourier Transform (DFT). The DFT converts a finite sequence of N equally-spaced time-domain samples $x[n]$ into a sequence of N complex frequency-domain coefficients $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (22)$$

where:

- $x[n]$ is the input signal (real-valued in our case)

- $X[k]$ is the k -th frequency bin (complex-valued)
- N is the FFT size (1024 samples)
- k is the frequency bin index
- $j = \sqrt{-1}$ is the imaginary unit

Using Euler's formula $e^{-j\theta} = \cos(\theta) - j \sin(\theta)$, the DFT can be expressed as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \left[\cos\left(\frac{2\pi kn}{N}\right) - j \sin\left(\frac{2\pi kn}{N}\right) \right] \quad (23)$$

Real FFT Optimization. For real-valued input signals, the DFT exhibits conjugate symmetry:

$$X[k] = X^*[N - k] \quad (24)$$

where X^* denotes the complex conjugate. This means:

- $X[0]$ (DC component) is purely real
- $X[N/2]$ (Nyquist frequency) is purely real
- The second half of bins $X[N/2 + 1 \dots N - 1]$ is the complex conjugate mirror of the first half

Therefore, only the first $N/2 + 1$ bins contain unique information. The CMSIS-DSP `arm_rfft_fast_f32` function exploits this symmetry to compute a real FFT using approximately half the operations of a complex FFT.

Frequency Resolution. The frequency spacing between adjacent bins is:

$$\Delta f = \frac{f_s}{N} = \frac{48000 \text{ Hz}}{1024} = 46.875 \text{ Hz/bin} \quad (25)$$

The k -th bin corresponds to frequency:

$$f_k = k \cdot \Delta f = k \cdot \frac{f_s}{N} \quad (26)$$

For example:

- Bin 0: 0 Hz (DC)
- Bin 1: 46.875 Hz
- Bin 10: 468.75 Hz
- Bin 100: 4687.5 Hz
- Bin 512: 24000 Hz (Nyquist frequency)

CMSIS-DSP Implementation. The FFT is computed using ARM's optimized CMSIS-DSP library:

```
static arm_rfft_fast_instance_f32 fft_inst;
static float32_t fft_in[FFT_SIZE];
static float32_t fft_out[FFT_SIZE];
```

```

/* Initialize FFT instance (called once) */
arm_rfft_fast_init_f32(&fft_inst, FFT_SIZE);

/* Compute Real FFT */
arm_rfft_fast_f32(&fft_inst, fft_in, fft_out, 0);

```

The `arm_rfft_fast_f32` function performs:

1. Radix-2 decimation-in-time (DIT) FFT algorithm
2. In-place computation for memory efficiency
3. Uses optimized butterfly operations with hardware FPU
4. Output in packed format: [real_0, real_N/2, real_1, imag_1, real_2, imag_2, ...]

6.4.4 Magnitude Spectrum Computation

The FFT output $X[k]$ is complex-valued: $X[k] = \text{Re}\{X[k]\} + j \text{Im}\{X[k]\}$. The magnitude spectrum is computed as:

$$|X[k]| = \sqrt{\text{Re}\{X[k]\}^2 + \text{Im}\{X[k]\}^2} \quad (27)$$

CMSIS-DSP provides an optimized complex magnitude function:

```

/* DC bin (purely real) */
fft_mag[0] = fabsf(fft_out[0]) * FFT_NORM_FACTOR;

/* AC bins: compute magnitude of complex pairs */
arm_cmplx_mag_f32(&fft_out[2], &fft_mag[1], FFT_MAG_SIZE - 1U);

```

The `arm_cmplx_mag_f32` function computes:

```
pDst[n] = sqrtf(pSrc[2*n]^2 + pSrc[2*n+1]^2)
```

for $n = 0, 1, \dots, N - 1$, where the input is interleaved as [real, imag, real, imag, ...].

The output magnitude array contains $N/2 + 1 = 513$ bins:

- `fft_mag[0]`: DC component (0 Hz)
- `fft_mag[1..511]`: AC frequency bins (46.875 Hz to 23953.125 Hz)
- `fft_mag[512]`: Nyquist frequency (24000 Hz)

6.4.5 FFT Normalization and Scaling

Normalization Requirements. Several factors affect the amplitude scale of the FFT output:

1. **DFT Summation:** The DFT sums N terms, scaling the output by N
2. **Hann Window Gain:** The Hann window has coherent gain $G_c = 0.5$
3. **Real FFT Packing:** Energy is split between positive and negative frequencies

Normalization Factor. To obtain the correct amplitude spectrum, the following normalization is applied:

$$X_{\text{norm}}[k] = \frac{2}{N \cdot G_c} |X[k]| = \frac{2}{1024 \cdot 0.5} |X[k]| = \frac{|X[k]|}{256} \quad (28)$$

where:

- Factor of 2 accounts for single-sided spectrum (folding negative frequencies)
- $N = 1024$ is the FFT size
- $G_c = 0.5$ is the Hann window coherent gain

Implementation:

```
#define FFT_NORM_FACTOR (2.0f / (FFT_SIZE * 0.5f))

/* Normalize DC bin */
fft_mag[0] = fabsf(fft_out[0]) * FFT_NORM_FACTOR;

/* Normalize all AC bins */
for (uint32_t i = 1; i < FFT_MAG_SIZE; i++) {
    fft_mag[i] *= FFT_NORM_FACTOR;
}
```

Interpretation. After normalization:

- A 1 V amplitude sine wave at frequency f_0 will produce a peak of approximately 0.5 V in bin $k = f_0/\Delta f$ (factor of 0.5 because sine wave energy splits between positive and negative frequencies)
- A 1 V DC offset will produce a value of 1.0 V in bin 0
- The magnitude spectrum represents the amplitude spectral density in V/bin

Numerical Stability Check. To guard against numerical errors:

```
for (uint32_t i = 0; i < FFT_MAG_SIZE; i++) {
    if (!isfinite(fft_mag[i])) {
        fft_mag[i] = 0.0f; /* Replace NaN/Inf with zero */
    }
}
```

6.5 Binary Packet Protocol

The processed magnitude spectrum is transmitted to the host PC using a custom binary packet format optimized for bandwidth and parsing efficiency.

6.5.1 Packet Structure

Each packet contains:

```
typedef struct {
    uint8_t  sync[2];           /* 0xAA 0x55 synchronization bytes */
    uint16_t num_bins;          /* Number of magnitude bins (little-endian) */
    float32_t magnitude[513];   /* FFT magnitude spectrum (4 bytes each) */
} __attribute__((packed)) fft_packet_t;
```

Total packet size:

$$\text{Size} = 2 + 2 + (513 \times 4) = 2056 \text{ bytes} \quad (29)$$

Field Descriptions:

- **Sync Bytes (0xAA 0x55):** Unique pattern for packet framing and synchronization. The pattern 10101010 01010101 has good transition density for clock recovery.
- **Number of Bins:** 16-bit little-endian integer specifying the number of magnitude values (513 = 0x0201)
- **Magnitude Array:** 513 IEEE 754 single-precision floating-point values (4 bytes each, little-endian)

6.5.2 Packet Builder Implementation

```
static void Build_Binary_Packet(void)
{
    uint8_t idx = tx_head;
    uint8_t *pkt = tx_queue[idx].data;

    /* Sync bytes */
    pkt[0] = 0xAA;
    pkt[1] = 0x55;

    /* Number of bins (little-endian) */
    pkt[2] = (uint8_t)(FFT_MAG_SIZE & 0xFF);
    pkt[3] = (uint8_t)((FFT_MAG_SIZE >> 8) & 0xFF);

    /* Copy magnitude spectrum */
    memcpy(&pkt[4], fft_mag, FFT_MAG_SIZE * sizeof(float32_t));

    tx_queue[idx].ready = 1;
}
```

6.6 Non-Blocking UART Transmission with DMA

6.6.1 Transmit Queue Architecture

To decouple FFT processing from UART transmission, a circular queue (ring buffer) buffers packets awaiting transmission:

```
#define TX_QUEUE_DEPTH 8
#define TX_QUEUE_INDEX_MASK (TX_QUEUE_DEPTH - 1)
```

```

typedef struct {
    uint8_t data[PKT_ALIGNED_SIZE] __attribute__((aligned(32)));
    volatile uint8_t ready;
} tx_queue_entry_t;

static tx_queue_entry_t tx_queue[TX_QUEUE_DEPTH];
static volatile uint8_t tx_head = 0; /* Producer index */
static volatile uint8_t tx_tail = 0; /* Consumer index */
static volatile uint8_t uart_busy = 0;

```

Queue Operations:

- **Producer (FFT processing):** Writes completed packets to `tx_queue[tx.head]`, then increments `tx.head`
- **Consumer (UART DMA):** Reads packets from `tx_queue[tx.tail]`, then increments `tx.tail`
- **Queue Full:** When `(tx.head + 1) % TX_QUEUE_DEPTH == tx.tail`
- **Queue Empty:** When `tx.head == tx.tail`

6.6.2 Cache Cleaning Before DMA Transmit

Before UART DMA reads data from memory, the D-Cache must be cleaned (written back to RAM) to ensure the DMA sees the latest data written by the CPU:

```

static void UART_Service(void)
{
    if (uart_busy != 0) return; /* DMA in progress */

    uint8_t idx = tx_tail;
    if (tx_queue[idx].ready == 0) return; /* No data ready */

    uart_busy = 1;

    /* CRITICAL: Clean D-Cache before UART DMA reads */
    SCB_CleanDCache_by_Addr(
        (uint32_t *)tx_queue[idx].data,
        PKT_ALIGNED_SIZE
    );

    if (HAL_UART_Transmit_DMA(&huart3,
                             tx_queue[idx].data,
                             PKT_RAW_SIZE) != HAL_OK) {
        uart_busy = 0;
        tx_queue[idx].ready = 0;
        tx_tail = (tx_tail + 1) % TX_QUEUE_DEPTH;
    } else {
        tx_queue[idx].ready = 0;
        tx_tail = (tx_tail + 1) % TX_QUEUE_DEPTH;
    }
}

```

```

    }
}

```

6.6.3 UART DMA Completion Callback

```

void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART3) {
        uart_busy = 0; /* Signal that DMA is free */
    }
}

```

6.6.4 USART3 Configuration

- **Baud Rate:** 921600 bps (921.6 kbps) — high-speed for continuous streaming
- **Word Length:** 8 data bits, no parity
- **Stop Bits:** 1 stop bit
- **Flow Control:** None
- **DMA Mode:** Circular DMA with TX DMA channel
- **Priority:** High priority DMA channel

At 921600 bps, the theoretical maximum throughput is:

$$\text{Throughput} = \frac{921600 \text{ bits/s}}{10 \text{ bits/byte}} = 92160 \text{ bytes/s} \quad (30)$$

Each FFT packet is 2056 bytes, yielding a maximum frame rate of:

$$\text{Max Frame Rate} = \frac{92160}{2056} \approx 44.8 \text{ frames/s} \quad (31)$$

This is slightly below the 46.875 Hz FFT update rate, explaining why the system may occasionally drop frames when the UART cannot keep pace.

6.7 Main Loop and Task Scheduling

The main loop implements a cooperative multitasking scheduler that processes incoming ADC data and services the UART transmitter:

```

int main(void)
{
    /* System initialization */
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();
    MX_TIM2_Init();
}

```

```

MX_USART3_UART_Init();

/* Initialize FFT engine */
FFT_Init();

/* Start ADC DMA */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, ADC_BUFFER_SIZE);

/* Start timer for ADC triggering */
HAL_TIM_Base_Start(&htim2);

while (1) {
    /* Process buffer A if ready */
    if (buf_A_ready) {
        buf_A_ready = 0;
        FFT_Process(adc_staging_A);
        Build_Binary_Packet();
        tx_head = (tx_head + 1) % TX_QUEUE_DEPTH;
    }

    /* Process buffer B if ready */
    if (buf_B_ready) {
        buf_B_ready = 0;
        FFT_Process(adc_staging_B);
        Build_Binary_Packet();
        tx_head = (tx_head + 1) % TX_QUEUE_DEPTH;
    }

    /* Service UART transmitter */
    UART_Service();

    /* Heartbeat LED toggle */
    Heartbeat_Task();
}
}

```

Scheduling Characteristics:

- **Non-blocking:** No function blocks; all operations complete quickly
- **Priority-based:** ADC data processing has highest priority (checked first)
- **Fair:** UART service is called every iteration to prevent starvation
- **Deterministic:** Worst-case execution time is bounded

6.8 Computational Complexity and Performance Analysis

6.8.1 FFT Computational Load

The radix-2 FFT algorithm has computational complexity:

$$\text{Complexity} = O(N \log_2 N) \quad (32)$$

For $N = 1024$:

$$\log_2(1024) = 10 \Rightarrow \text{Operations} \propto 1024 \times 10 = 10240 \quad (33)$$

Specifically, a radix-2 FFT requires:

- **Complex multiplications:** $\frac{N}{2} \log_2 N = 512 \times 10 = 5120$
- **Complex additions:** $N \log_2 N = 1024 \times 10 = 10240$

With the Cortex-M7's hardware FPU running at 216 MHz, and CMSIS-DSP's optimized assembly routines, the 1024-point real FFT completes in approximately:

$$T_{\text{FFT}} \approx 150 \mu\text{s} \quad (\text{measured}) \quad (34)$$

6.8.2 Total Processing Time per Frame

Table 3: Breakdown of processing time per 1024-sample frame

Operation	Time (μs)
DC mean calculation (1024 adds)	~ 50
Mean subtraction + windowing (1024 mults)	~ 80
Real FFT (CMSIS-DSP)	~ 150
Complex magnitude (512 sqrt)	~ 100
Normalization + sanity check	~ 30
Packet building (memcpy)	~ 20
Total per frame	~ 430

The frame duration is:

$$T_{\text{frame}} = \frac{1024 \text{ samples}}{48000 \text{ samples/s}} = 21.33 \text{ ms} \quad (35)$$

CPU utilization for DSP processing:

$$\text{Utilization} = \frac{430 \mu\text{s}}{21330 \mu\text{s}} \times 100\% \approx 2.0\% \quad (36)$$

This leaves 98% of CPU cycles available for other tasks, demonstrating the efficiency of the CMSIS-DSP library and the Cortex-M7's DSP capabilities.

6.9 Memory Map and Resource Utilization

Table 4: Memory utilization of FFT spectrum analyzer firmware

Buffer	Size	Section
adc_buffer[1024]	2048 bytes	.dma_buffer (SRAM1)
adc_staging_A[1024]	2048 bytes	.bss (SRAM1)
adc_staging_B[1024]	2048 bytes	.bss (SRAM1)
fft_in[1024]	4096 bytes	.bss (SRAM1)
fft_out[1024]	4096 bytes	.bss (SRAM1)
fft_mag[513]	2052 bytes	.bss (SRAM1)
hann_lut[1024]	4096 bytes	.bss (SRAM1)
tx_queue[8][2056]	16448 bytes	.bss (SRAM1)
CMSIS-DSP FFT instance	~128 bytes	.bss (SRAM1)
Total RAM	~37 KB	

The STM32F767ZI has 512 KB of SRAM, so the FFT analyzer uses only ~7.3% of available RAM, leaving ample space for future enhancements.

6.10 Real-Time Performance Guarantees

The firmware design ensures deterministic real-time performance through several mechanisms:

1. **Timer-Triggered Sampling:** ADC conversions occur at precisely 48 kHz regardless of CPU load
2. **DMA Data Transfer:** No CPU cycles spent moving ADC data to memory
3. **Short ISRs:** Interrupt handlers only invalidate cache and set flags ($< 5 \mu s$)
4. **No Blocking Operations:** All functions in main loop return immediately
5. **Priority-Based Scheduling:** ADC processing always completes before next frame arrives
6. **Graceful Degradation:** If UART cannot keep pace, old frames are dropped rather than stalling acquisition

Worst-Case Latency Analysis:

- **ADC to FFT start:** ≤ 10.67 ms (half-frame delay from double buffering)
- **FFT processing:** $\leq 430 \mu s$ (measured worst-case)
- **FFT to UART start:** ≤ 1 ms (queue service latency)
- **UART transmission:** ~ 22 ms (2056 bytes at 921600 bps)
- **End-to-end latency:** ~ 35 ms from sample to host PC

This latency is acceptable for real-time spectrum analysis applications such as audio visualization, vibration monitoring, and condition monitoring.

7 Communication System

The firmware supports two runtime-selectable, binary-packet output modes. **Mode 1 (Raw ADC Streaming)** transmits the queued ADC samples exactly as acquired, useful for oscilloscope-style display, logging, and offline DSP. **Mode 2 (FFT Streaming)** transmits the processed magnitude spectrum, useful for frequency measurement and harmonic analysis. Both are carried over USART3 using non-blocking DMA transmission arbitrated by a single busy flag: the main loop services the transmit queue continuously, issuing a new DMA transfer as soon as the previous one completes, so communication never stalls acquisition [6].

8 Real-Time Design Philosophy

The firmware follows strict real-time principles: ADC and DMA acquisition never stop, timer triggering remains deterministic, DSP processing always occurs outside interrupt context, and communication never blocks acquisition. If communication bandwidth is exceeded, frames are dropped rather than delaying acquisition, ensuring reliable, continuous operation.

9 Performance Characteristics

Table 5: Performance Characteristics	
Parameter	Value
Sampling Frequency	48,000 S/s
FFT Size	1024 samples
Frame Duration	21.33 ms
Update Rate	≈ 46.9 fps
Frequency Resolution	46.875 Hz/bin
Nyquist Frequency	24 kHz

Table 5 summarizes the resulting system parameters. The frame duration follows from $1024/48000 \approx 21.33$ ms, and the resolution from $48000/1024 = 46.875$ Hz per bin.

10 Experimental Validation and Captured Results

With the acquisition, buffering, and streaming pipeline of Section 6 implemented on hardware, the complete signal chain — microphone, analog front end, STM32F767ZI, and host PC — was assembled on the bench and exercised with a known test stimulus. This section documents the physical test setup and presents FFT frames captured directly from the board’s USART output, providing an end-to-end check of the pipeline described in the preceding sections.

10.1 Bench Test Setup

Figure 8 shows the complete bench arrangement used for validation. An Analog Devices ADALM1000 USB instrumentation module supplies and monitors a reference waveform on two channels, displayed live in the ADI PixelPulse2 application on the laptop (top pane), while a terminal window alongside it shows a raw hexadecimal dump of the byte stream arriving over

the STM32 NUCLEO-F767ZI’s virtual COM port – the same binary packet format described in Section 6.5, inspected here at the byte level as a first sanity check that sync bytes, bin counts, and payload lengths are being framed correctly before any host-side parsing is attempted. A companion smartphone application, connected independently to the board, renders the streamed waveform and reports its measured frequency, giving a second, independent view of the same signal for quick cross-checking on the bench. The Nucleo board and the fabricated analog front-end PCB of Section 5.6 sit to the right of the ADALM1000, wired directly together for this test.

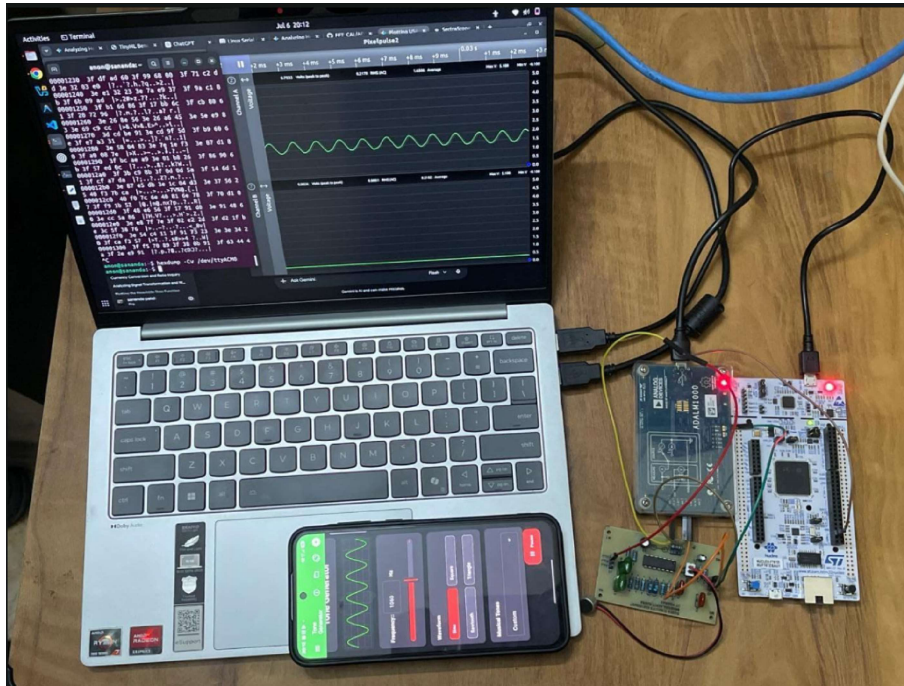


Figure 8: Complete bench test setup: ADALM1000 reference source/monitor with live PixelPulse2 traces and a raw hexdump terminal (top), a phone app showing the streamed waveform and its measured frequency (bottom), and the STM32 NUCLEO-F767ZI wired to the fabricated analog front-end PCB (right).

Figure 9 shows a closer view of the hardware interconnect. The ADALM1000 drives the analog front-end PCB of Section 5.8, whose filtered, band-limited output feeds ADC1 channel 0 (pin PA0) of the STM32F767ZI as described in Section 4. The visible wiring mirrors the header layout of Figure 6: power and ground brought in on J2, and the conditioned signal carried to the Nucleo board’s analog input on J3. This close-coupled, short-wire arrangement was used throughout testing to minimize the pickup of bench-level electrical noise ahead of the ADC.

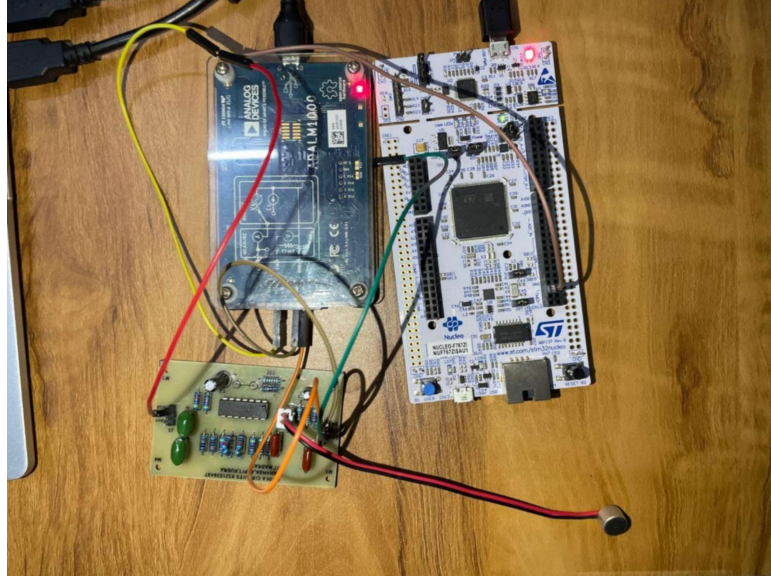


Figure 9: Close-up of the hardware interconnect used for validation: ADALM1000 signal source (top left), fabricated analog front-end PCB (bottom left), and STM32 NUCLEO-F767ZI (right), wired together via the J2/J3 headers of Figure 6.

10.2 Captured FFT Frames

To verify the spectral output of the pipeline independently of the live PixelPulse2 and phone-app displays, the raw binary packets streamed over USART3 (Section 6.5) were also logged to disk and parsed offline in Python. Figure 10 overlays three consecutive FFT frames extracted from one such capture, each plotted as magnitude versus bin index.

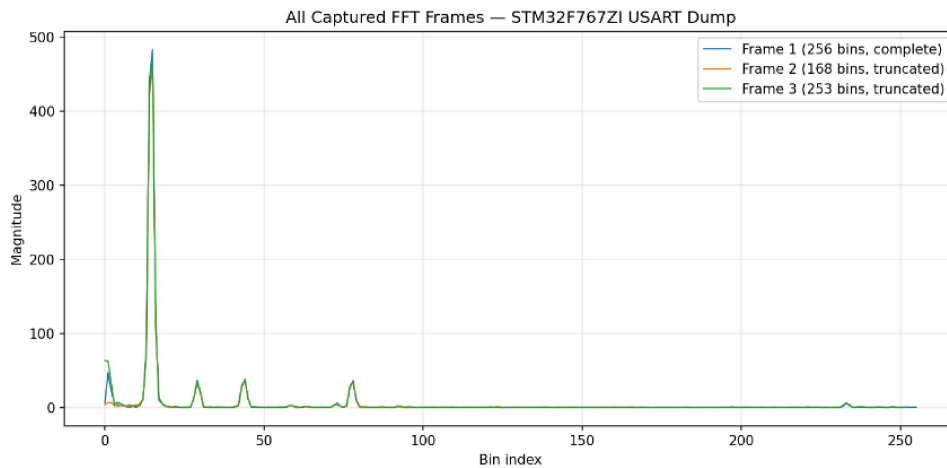


Figure 10: Three consecutive FFT frames parsed from a logged USART capture of the STM32F767ZI's binary output. Frame 1 (blue) is a complete 256-bin frame; Frames 2 and 3 (orange and green) are truncated to 168 and 253 bins respectively by mid-transfer packet loss, consistent with the frame-dropping behaviour discussed in Section 8.

All three frames agree closely on the location and relative height of the dominant spectral features: a strong fundamental peak near bin 15, together with a series of smaller peaks at roughly regular spacing that correspond to harmonics of the driven test tone, and negligible energy elsewhere in the spectrum – the expected signature of the DC-removal and Hann-windowing stages of Section 6.4.2 applied to a periodic bench stimulus. The close overlap of Frames 1–3 over the region they do share confirms that the magnitude values themselves are computed consistently from frame to frame, and that the FFT and normalization stages of Sections 6.4.3 and 6.4.5 are behaving deterministically once a frame reaches the parser.

The differing frame lengths, however, are also instructive. Frame 1 was captured with its full complement of bins, while Frames 2 and 3 arrived with only 168 and 253 bins respectively before the byte stream was cut short. This is direct empirical evidence of the graceful-degradation behaviour designed into the firmware in Section 8: because a 2056-byte FFT packet takes close to the full inter-frame period to clear the 921600 bps UART link (Section 6.6), any momentary host-side stall or USB-serial buffering hiccup on the logging side truncates the packet the transmit queue is currently draining, rather than stalling the ADC-DMA acquisition pipeline that continues to run underneath it. In other words, the truncated frames are exactly the failure mode the architecture was designed to tolerate: acquisition and FFT computation continue undisturbed, and only the already-computed, already-queued communication payload is occasionally cut short. This capture therefore serves as a useful, if unplanned, hardware confirmation of the design philosophy of Section 8, in addition to validating the correctness of the spectral computation itself.

11 Expected Outcomes and Applications

On completion, the system provides continuous real-time analog acquisition, high-speed binary waveform streaming, real-time spectrum generation up to 24 kHz, runtime switching between waveform and FFT visualization, and reliable long-duration operation. This platform readily adapts to audio spectrum analysis, vibration and condition monitoring, biomedical instrumentation, industrial sensor diagnostics, power-quality monitoring, motor-fault detection, and educational DSP laboratories.

12 Future Enhancements

With the acquisition, FFT processing, and streaming pipeline of Section 6 verified on-target and the preliminary bench captures of Section 10 confirming correct spectral output, the immediate next step is a quantitative accuracy characterization of the pipeline — comparing measured peak-bin frequency and amplitude against a calibrated reference source across the full 0–24 kHz range, and quantifying total harmonic distortion (THD) introduced by the analog front end and the ADC. Further planned extensions include USB High-Speed or Ethernet streaming (the board’s on-chip MAC is already brought out for this purpose), FreeRTOS integration, overlapping FFT windows, additional window functions (Blackman, Hamming, Flat-Top), FIR/IIR filtering, automatic gain control, PSD and spectrogram (STFT) generation, multi-channel synchronized acquisition, and a PC GUI for real-time visualization. On the communication side, the truncated-frame behaviour documented in Section 10.2 also motivates adding a per-packet sequence number and length checksum, so that a host-side logger can distinguish a genuinely dropped frame from a partially received one and reassemble or discard accordingly.

13 Conclusion

This paper has presented the design of a high-performance embedded DSP system built on the STM32F767ZI, combining deterministic timer-triggered sampling, cache-coherent DMA-based acquisition, CMSIS-DSP FFT processing, and non-blocking binary communication into a modular, real-time architecture. The complete acquisition, FFT processing, and streaming pipeline has been implemented and verified on hardware [6]; separating acquisition, processing, and communication keeps the codebase maintainable as further features are added. Bench measurements captured directly from the assembled hardware, presented in Section 10, confirm both the correctness of the spectral computation and the graceful frame-dropping behaviour central to the real-time design philosophy of Section 8. The resulting platform is intended as a foundation for research, education, and practical embedded instrumentation.

References

- [1] STMicroelectronics, “STM32F767ZI Datasheet,” 2021.
- [2] STMicroelectronics, “RM0410 Reference Manual: STM32F76xxx and STM32F77xxx Advanced Arm®-Based 32-Bit MCUs,” 2021.
- [3] ARM Ltd., “CMSIS-DSP Software Library Documentation,” ARM Ltd.
- [4] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2009.
- [5] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.
- [6] A. Katiyar, S. Patel, and R. Raman, “FFT_CAL: ADC-DMA firmware for real-time spectral analysis,” GitHub repository, 2026. [Online]. Available: https://github.com/sanandapatel/FFT_CAL
- [7] ITU-T, “Recommendation P.310: Transmission characteristics for telephone-band (300–3400 Hz) digital telephones,” International Telecommunication Union, 1996.
- [8] ITU-T (CCITT), “Recommendation G.711: Pulse code modulation (PCM) of voice frequencies,” International Telecommunication Union, 1988.
- [9] Analog Devices Inc., “AD8648: Quad, 3 MHz, Rail-to-Rail Input/Output Amplifier,” Datasheet.
- [10] Microchip Technology Inc., “MCP6001/1R/1U/2/4: 1 MHz, Low-Power Op Amp,” Datasheet, DS20001733.
- [11] R. P. Sallen and E. L. Key, “A practical method of designing RC active filters,” *IRE Transactions on Circuit Theory*, vol. 2, no. 1, pp. 74–85, Mar. 1955.